Name: Mustansir Rangwala

Batch: B2

Problem Statement: Perform bag-of-words approach (count occurrence, normalized count occurrence), tf-idf on data. Create embeddings using Word2Vec.

In [74]:
```
import pandas as pd
import numpy as np
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectori
from gensim.models import Word2Vec
```

In [75]:
```
data = pd.read_csv('../Data/assignment2.csv')
```

In [76]:
```
data.head()
```

Out[76]:

	Make	Model	Year	Engine Fuel Type	Engine HP	Engine Cylinders	Transmission Type	Driven_Wheels
0	BMW	1 Series M	2011	premium unleaded (required)	335.0	6.0	MANUAL	rear wheel drive
1	BMW	1 Series	2011	premium unleaded (required)	300.0	6.0	MANUAL	rear wheel drive
2	BMW	1 Series	2011	premium unleaded (required)	300.0	6.0	MANUAL	rear wheel drive
3	BMW	1 Series	2011	premium unleaded (required)	230.0	6.0	MANUAL	rear wheel drive
4	BMW	1 Series	2011	premium unleaded (required)	230.0	6.0	MANUAL	rear wheel drive

In [77]:
```
data.describe(include='all')
```

Out[77]:

	Make	Model	Year	Engine Fuel Type	Engine HP	Engine Cylinders	Tı
count	11914	11914	11914.000000	11911	11845.00000	11884.000000	
unique	48	915	NaN	10	NaN	NaN	
top	Chevrolet	Silverado 1500	NaN	regular unleaded	NaN	NaN	
freq	1123	156	NaN	7172	NaN	NaN	
mean	NaN	NaN	2010.384338	NaN	249.38607	5.628829	
std	NaN	NaN	7.579740	NaN	109.19187	1.780559	
min	NaN	NaN	1990.000000	NaN	55.00000	0.000000	
25%	NaN	NaN	2007.000000	NaN	170.00000	4.000000	
50%	NaN	NaN	2015.000000	NaN	227.00000	6.000000	
75%	NaN	NaN	2016.000000	NaN	300.00000	6.000000	
max	NaN	NaN	2017.000000	NaN	1001.00000	16.000000	

In [78]:

```
#Removing the NAN Values
data.dropna(inplace=True)
data.reset_index(inplace=True)
data.isnull().sum()
```

Out[78]:

```
index                0
Make                 0
Model                0
Year                 0
Engine Fuel Type     0
Engine HP            0
Engine Cylinders     0
Transmission Type    0
Driven_Wheels        0
Number of Doors      0
Market Category      0
Vehicle Size         0
Vehicle Style        0
highway MPG          0
city mpg             0
Popularity           0
MSRP                 0
dtype: int64
```

Preprocessing the by data by removing stopwords and tokenizing

In [79]:

```
stop_words = set(stopwords.words('english'))

def preprocess(text):
    tokens = word_tokenize(text.lower())
    tokens = [token for token in tokens if token.isalpha() and token not
    return tokens
```

```
data['Market Category'] = data['Market Category'].apply(preprocess)

data['Market Category']
```

Out[79]:

```
0              [factory, tuner, luxury]
1                 [luxury, performance]
2                              [luxury]
3                 [luxury, performance]
4                              [luxury]
                     ...
8079    [crossover, hatchback, luxury]
8080    [crossover, hatchback, luxury]
8081    [crossover, hatchback, luxury]
8082    [crossover, hatchback, luxury]
8083                           [luxury]
Name: Market Category, Length: 8084, dtype: object
```

Perform the bag-of-words approach using CountVectorizer

In [80]:

```
count_vectorizer = CountVectorizer(tokenizer=lambda doc: doc, lowercase=F
bow = count_vectorizer.fit_transform(data['Market Category'])
print(bow.toarray()[:5, :10])
```

```
[[0 0 0 1 0 0 0 0 1 0]
 [0 0 0 0 0 0 0 0 1 1]
 [0 0 0 0 0 0 0 0 1 0]
 [0 0 0 0 0 0 0 0 1 1]
 [0 0 0 0 0 0 0 0 1 0]]
```

- We can also calculate the normalized count occurrence by dividing the count by the length of each document

In [81]:

```
normalized_count = bow.copy()
for i, j in zip(*normalized_count.nonzero()):
    normalized_count[i, j] = normalized_count[i, j] / len(data['Market Ca
print(normalized_count.toarray()[:5, :10]) # Print the first 5 rows and 1
```

```
[[0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 1 0]
 [0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 1 0]]
```

TF-IDF to calculate the importance of each word in the document

In [82]:

```
tfidf_vectorizer = TfidfVectorizer(tokenizer=lambda doc: doc, lowercase=F
tfidf = tfidf_vectorizer.fit_transform(data['Market Category'])
print(tfidf.toarray()[:5, :10]) # Print the first 5 rows and 10 columns o
```

```
[[0.         0.         0.         0.6616931  0.         0.
  0.         0.         0.35259679 0.        ]
 [0.         0.         0.         0.         0.         0.
  0.         0.         0.62784336 0.77833972]
 [0.         0.         0.         0.         0.         0.
  0.         0.         1.         0.        ]
 [0.         0.         0.         0.         0.         0.
  0.         0.         0.62784336 0.77833972]
 [0.         0.         0.         0.         0.         0.
  0.         0.         1.         0.        ]]
```

Creating word embeddings using Word2Vec

In [83]:
```
model = Word2Vec(data['Market Category'], min_count=1)
embeddings = np.zeros((len(data), 100))
for i, tokens in enumerate(data['Market Category']):
    for token in tokens:
        embeddings[i] += model.wv[token]
    embeddings[i] /= len(tokens)
```

```
C:\Users\gv7lo\AppData\Local\Temp\ipykernel_6352\3049100346.py:6: RuntimeW
arning: invalid value encountered in true_divide
  embeddings[i] /= len(tokens)
```

In [84]:
```
embeddings
```

Out[84]:
```
array([[-0.01106373,  0.00585934,  0.01540476, ..., -0.03831561,
         0.01001132,  0.00379507],
       [-0.01955247,  0.00990319,  0.01476101, ..., -0.03403803,
         0.00382115,  0.00643379],
       [-0.0184865 ,  0.00986475,  0.01654671, ..., -0.04207258,
         0.0108053 ,  0.00743209],
       ...,
       [-0.01277368,  0.00704898,  0.00513679, ..., -0.02371885,
         0.00932181, -0.00200806],
       [-0.01277368,  0.00704898,  0.00513679, ..., -0.02371885,
         0.00932181, -0.00200806],
       [-0.0184865 ,  0.00986475,  0.01654671, ..., -0.04207258,
         0.0108053 ,  0.00743209]])
```

- We have successfully created bag-of-words vectors, normalized count occurrence vectors, TF-IDF vectors, and word embeddings for the dataset.